

语言教学预约系统后端API程序实现（标注对应设计项目）

说明：本程序基于设计说明文档「二、API设计」模块实现，采用Java语言（Spring Boot + Spring Security + MyBatis），严格遵循RESTful风格、权限控制、数据校验等设计规则，每处功能均标注对应设计文档中的具体项目，确保设计与实现一致。

一、项目基础配置（对应设计2.1 设计概述）

1.1 核心依赖配置（pom.xml关键依赖）

```
1  <!-- 基础依赖 -->
2  <dependency>
3      <groupId>org.springframework.boot</groupId>
4      <artifactId>spring-boot-starter-web</artifactId>
5  </dependency>
6  <!-- 数据校验（对应设计2.4 异常处理、参数校验） -->
7  <dependency>
8      <groupId>org.springframework.boot</groupId>
9      <artifactId>spring-boot-starter-validation</artifactId>
10 </dependency>
11 <!-- 权限控制（对应设计2.3 安全设计-RBAC模型） -->
12 <dependency>
13     <groupId>org.springframework.boot</groupId>
14     <artifactId>spring-boot-starter-security</artifactId>
15 </dependency>
16 <!-- JWT Token（对应设计2.3 安全设计-Token加密） -->
17 <dependency>
18     <groupId>io.jsonwebtoken</groupId>
19     <artifactId>jjwt</artifactId>
20     <version>0.9.1</version>
21 </dependency>
22 <!-- 数据库相关 -->
23 <dependency>
24     <groupId>org.mybatis.spring.boot</groupId>
25     <artifactId>mybatis-spring-boot-starter</artifactId>
26 </dependency>
27 <dependency>
28     <groupId>mysql</groupId>
```

```

29     <artifactId>mysql-connector-java</artifactId>
30     <scope>runtime</scope>
31 </dependency>
32 <!-- lombok依赖（实体类@Data注解生效，对应所有entity实体类） -->
33 <dependency>
34     <groupId>org.projectlombok</groupId>
35     <artifactId>lombok</artifactId>
36     <optional>true</optional>
37 </dependency>
38 <!-- 支付相关依赖（对应2.3 学生选课缴费模块） -->
39 <dependency>
40     <groupId>com.alipay.sdk</groupId>
41     <artifactId>alipay-sdk-java</artifactId>
42     <version>4.34.0.ALL</version>
43 </dependency>

```

1.2 全局配置（application.yml）

```

1  server:
2    port: 8080
3    servlet:
4      context-path: /api/v1 # 对应设计2.1 基础配置-接口基础路径
5  spring:
6    datasource:
7      driver-class-name: com.mysql.cj.jdbc.Driver
8      url: jdbc:mysql://localhost:3306/language_reservation?
9        useUnicode=true&characterEncoding=utf-8&serverTimezone=GMT%2B8
10     username: root
11     password: 123456
12  mybatis:
13    mapper-locations: classpath:mapper/**/*.xml
14    type-aliases-package: com.language.reservation.entity
15  # JWT配置（对应设计2.3 安全设计-Token）
16  jwt:
17    secret: language_reservation_system_2026 # 加密密钥
18    expiration: 604800000 # Token有效期7天（对应设计2.2.1 登录接口说明）
19  # 接口频率限制（对应设计2.3 安全设计-接口防护）
20  rate-limit:
21    enabled: true
22    limit: 100 # 每分钟最大请求数
23  # 支付宝支付配置（对应2.3 学生选课缴费模块-支付功能）
24  alipay:
25    app-id: 202100000000000000 # 支付宝应用ID（模拟）
26    private-key: MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAKcwggSjAgEAAoIBAQ... # 应用私
27    钥（模拟）

```

```
26     public-key: MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA... # 支付宝公钥（模  
拟）  
27     gateway-url: https://openapi.alipaydev.com/gateway.do # 支付宝沙箱网关（开发环  
境）  
28     return-url: http://localhost:8080/api/v1/reservation/pay/return # 支付同步回  
调地址  
29     notify-url: http://localhost:8080/api/v1/reservation/pay/notify # 支付异步回  
调地址
```

1.3 全局统一返回格式（对应设计2.1 基础配置-返回格式）

```
1  package com.language.reservation.common;  
2  
3  import lombok.Data;  
4  
5  /**  
6   * 统一返回结果封装，对应设计2.1中“返回格式: code、message、data”  
7   */  
8  @Data  
9  public class Result<T> {  
10     // 状态码（对应设计2.1 状态码规范）  
11     private Integer code;  
12     // 提示信息  
13     private String message;  
14     // 返回数据（可选）  
15     private T data;  
16  
17     // 成功返回（无数据）  
18     public static <T> Result<T> success() {  
19         Result<T> result = new Result<>();  
20         result.setCode(200); // 200-成功（对应设计2.1）  
21         result.setMessage("操作成功");  
22         return result;  
23     }  
24  
25     // 成功返回（有数据）  
26     public static <T> Result<T> success(T data, String message) {  
27         Result<T> result = new Result<>();  
28         result.setCode(200);  
29         result.setMessage(message);  
30         result.setData(data);  
31         return result;  
32     }  
33  
34     // 失败返回（对应设计2.4 异常处理）
```

```

35     public static <T> Result<T> fail(Integer code, String message) {
36         Result<T> result = new Result<>();
37         result.setCode(code); // 400/401/403/404/500 (对应设计2.1)
38         result.setMessage(message);
39         return result;
40     }
41 }

```

1.4 全局异常处理（对应设计2.4 异常处理）

```

1  package com.language.reservation.exception;
2
3  import com.language.reservation.common.Result;
4  import org.springframework.validation.BindingResult;
5  import org.springframework.validation.FieldError;
6  import org.springframework.web.bind.MethodArgumentNotValidException;
7  import org.springframework.web.bind.annotation.ExceptionHandler;
8  import org.springframework.web.bind.annotation.RestControllerAdvice;
9
10 /**
11  * 全局异常处理，对应设计2.4 异常处理机制，统一拦截所有异常并返回规范格式
12  */
13 @RestControllerAdvice
14 public class GlobalExceptionHandler {
15
16     // 参数校验异常（400），对应设计2.4 参数异常
17     @ExceptionHandler(MethodArgumentNotValidException.class)
18     public Result<Void>
19     handleValidationException(MethodArgumentNotValidException e) {
20         BindingResult bindingResult = e.getBindingResult();
21         FieldError fieldError = bindingResult.getFieldError();
22         String message = fieldError != null ? fieldError.getDefaultMessage()
23         : "参数错误";
24         return Result.fail(400, message); // 400-参数错误（对应设计2.1）
25     }
26
27     // 未登录/Token失效（401），对应设计2.4 权限异常
28     @ExceptionHandler(UnLoginException.class)
29     public Result<Void> handleUnLoginException(UnLoginException e) {
30         return Result.fail(401, e.getMessage()); // 401-未登录（对应设计2.1）
31     }
32
33     // 权限不足（403），对应设计2.4 权限异常
34     @ExceptionHandler(NoPermissionException.class)
35     public Result<Void> handleNoPermissionException(NoPermissionException e) {

```

```

34         return Result.fail(403, e.getMessage()); // 403-权限不足 (对应设计2.1)
35     }
36
37     // 资源不存在 (404) , 对应设计2.4 资源异常
38     @ExceptionHandler(ResourceNotFoundException.class)
39     public Result<Void>
40     handleResourceNotFoundException(ResourceNotFoundException e) {
41         return Result.fail(404, e.getMessage()); // 404-资源不存在 (对应设计
42         2.1)
43     }
44
45     // 业务异常 (400) , 对应设计2.4 业务异常
46     @ExceptionHandler(BusinessException.class)
47     public Result<Void> handleBusinessException(BusinessException e) {
48         return Result.fail(400, e.getMessage());
49     }
50
51     // 服务器异常 (500) , 对应设计2.4 服务器异常
52     @ExceptionHandler(Exception.class)
53     public Result<Void> handleException(Exception e) {
54         e.printStackTrace(); // 记录异常日志 (对应设计2.3 安全设计-日志记录)
55         return Result.fail(500, "服务器繁忙, 请稍后再试"); // 500-服务器异常 (对应
56         设计2.1)
57     }
58 }

```

1.5 自定义异常类 (对应设计2.4 异常处理)

```

1  package com.language.reservation.exception;
2
3  /**
4   * 未登录异常 (401) , 对应设计2.4 权限异常
5   */
6  public class UnLoginException extends RuntimeException {
7      public UnLoginException(String message) {
8          super(message);
9      }
10 }
11
12 /**
13 * 权限不足异常 (403) , 对应设计2.4 权限异常
14 */
15 public class NoPermissionException extends RuntimeException {
16     public NoPermissionException(String message) {
17         super(message);
18     }
19 }

```

```

18     }
19 }
20
21 /**
22  * 资源不存在异常 (404) , 对应设计2.4 资源异常
23  */
24 public class ResourceNotFoundException extends RuntimeException {
25     public ResourceNotFoundException(String message) {
26         super(message);
27     }
28 }
29
30 /**
31  * 业务异常 (400) , 对应设计2.4 业务异常
32  */
33 public class BusinessException extends RuntimeException {
34     public BusinessException(String message) {
35         super(message);
36     }
37 }

```

1.6 JWT工具类（对应设计2.3 安全设计-Token加密）

```

1  package com.language.reservation.utils;
2
3  import io.jsonwebtoken.Claims;
4  import io.jsonwebtoken.Jwts;
5  import io.jsonwebtoken.SignatureAlgorithm;
6  import org.springframework.beans.factory.annotation.Value;
7  import org.springframework.stereotype.Component;
8
9  import java.util.Date;
10 import java.util.HashMap;
11 import java.util.Map;
12
13 /**
14  * JWT Token工具类, 对应设计2.3 安全设计-Token加密, 用于生成和解析Token
15  */
16 @Component
17 public class JwtUtil {
18
19     // 从配置文件读取JWT密钥和有效期
20     @Value("${jwt.secret}")
21     private String secret;
22     @Value("${jwt.expiration}")

```

```
23     private Long expiration;
24
25     /**
26      * 生成Token, 对应设计2.2.1 登录接口, 返回userId和role信息
27      */
28     public String generateToken(String userId, String role) {
29         Map<String, Object> claims = new HashMap<>();
30         claims.put("userId", userId); // 存储用户ID
31         claims.put("role", role);     // 存储角色 (student/teacher/admin)
32         // 生成Token, 设置有效期
33         return Jwts.builder()
34             .setClaims(claims)
35             .setExpiration(new Date(System.currentTimeMillis() +
expiration))
36             .signWith(SignatureAlgorithm.HS256, secret)
37             .compact();
38     }
39
40     /**
41      * 解析Token, 获取Claims (包含userId和role)
42      */
43     public Claims parseToken(String token) {
44         try {
45             return Jwts.parser()
46                 .setSigningKey(secret)
47                 .parseClaimsJws(token)
48                 .getBody();
49         } catch (Exception e) {
50             // Token失效/解析失败, 抛出未登录异常
51             throw new UnLoginException("Token失效或未登录");
52         }
53     }
54
55     /**
56      * 从Token中获取用户ID
57      */
58     public String getUserIdFromToken(String token) {
59         Claims claims = parseToken(token);
60         return claims.get("userId", String.class);
61     }
62
63     /**
64      * 从Token中获取角色信息
65      */
66     public String getRoleFromToken(String token) {
67         Claims claims = parseToken(token);
68         return claims.get("role", String.class);
```

```
69     }
70 }
```

1.7 PermissionCheck工具类（完整实现，对应权限校验逻辑）

```
1  package com.language.reservation.utils;
2
3  import com.language.reservation.exception.NoPermissionException;
4  import com.language.reservation.exception.UnLoginException;
5  import com.language.reservation.mapper.UserMapper;
6  import com.language.reservation.entity.User;
7  import org.springframework.beans.factory.annotation.Autowired;
8  import org.springframework.stereotype.Component;
9
10 /**
11  * PermissionCheck工具类，核心实现接口权限校验，对应设计2.3 安全设计-RBAC模型
12  * 被CourseController、StudentReservationController注入使用，实现角色校验和权限控制
13  */
14 @Component
15 public class PermissionCheck {
16
17     @Autowired
18     private JwtUtil jwtUtil;
19     @Autowired
20     private UserMapper userMapper;
21
22     /**
23      * 校验：仅管理员可操作（对应CourseController创建课程模板等接口）
24      * @param token 请求头中的Authorization Token
25      */
26     public void checkAdmin(String token) {
27         // 解析Token中的角色
28         String role = jwtUtil.getRoleFromToken(token);
29         // 校验角色是否为admin，否则抛出权限不足异常
30         if (!"admin".equals(role)) {
31             throw new NoPermissionException("您无管理员权限，无法执行该操作");
32         }
33     }
34
35     /**
36      * 校验：仅教师可操作（对应CourseController创建课程、设置排期等接口）
37      * @param token 请求头中的Authorization Token
38      */
39     public void checkTeacher(String token) {
```



```

40         // 解析Token中的角色和用户ID
41         String role = jwtUtil.getRoleFromToken(token);
42         String userId = jwtUtil.getUserIdFromToken(token);
43         // 1. 校验角色为teacher
44         if (!"teacher".equals(role)) {
45             throw new NoPermissionException("您无教师权限，无法执行该操作");
46         }
47         // 2. 校验教师账号状态为active (对应设计2.2.1 教师注册审核逻辑)
48         User teacher = userMapper.selectById(userId);
49         if (teacher == null || !"active".equals(teacher.getStatus())) {
50             throw new NoPermissionException("教师账号未审核或已冻结，无法执行操
51         }
52     }
53
54     /**
55     * 校验：仅学生可操作 (对应StudentReservationController所有学生接口)
56     * @param token 请求头中的Authorization Token
57     */
58     public void checkStudent(String token) {
59         // 解析Token中的角色和用户ID
60         String role = jwtUtil.getRoleFromToken(token);
61         String userId = jwtUtil.getUserIdFromToken(token);
62         // 1. 校验角色为student
63         if (!"student".equals(role)) {
64             throw new NoPermissionException("您无学生权限，无法执行该操作");
65         }
66         // 2. 校验学生账号状态为active
67         User student = userMapper.selectById(userId);
68         if (student == null || !"active".equals(student.getStatus())) {
69             throw new NoPermissionException("学生账号未激活或已冻结，无法执行操
70         }
71     }
72
73     /**
74     * 校验：教师或管理员可操作 (对应CourseController查询课程模板列表等接口)
75     * @param token 请求头中的Authorization Token
76     */
77     public void checkTeacherOrAdmin(String token) {
78         String role = jwtUtil.getRoleFromToken(token);
79         String userId = jwtUtil.getUserIdFromToken(token);
80         // 校验角色为admin，或角色为teacher且账号状态active
81         if ("admin".equals(role)) {
82             return; // 管理员直接通过
83         } else if ("teacher".equals(role)) {
84             User teacher = userMapper.selectById(userId);

```

```

85         if (teacher != null && "active".equals(teacher.getStatus())) {
86             return; // 教师账号正常, 通过校验
87         }
88     }
89     // 以上条件均不满足, 抛出权限不足异常
90     throw new NoPermissionException("您无权限执行该操作");
91 }
92
93 /**
94  * 辅助方法: 从Token中获取用户ID, 用于订单归属、课程归属校验
95  * @param token 请求头中的Authorization Token
96  * @return 用户ID (userId)
97  */
98 public String getUserIdFromToken(String token) {
99     try {
100         return jwtUtil.getUserIdFromToken(token);
101     } catch (Exception e) {
102         throw new UnLoginException("Token解析失败, 请重新登录");
103     }
104 }
105
106 /**
107  * 辅助校验: 教师创建课程/排期时, 校验课程归属 (课程的teacherId与Token中userId一
致)
108  * @param token Token
109  * @param teacherId 课程/排期中的教师ID
110  */
111 public void checkTeacherOwner(String token, String teacherId) {
112     // 校验教师权限
113     checkTeacher(token);
114     // 获取Token中的教师ID
115     String currentTeacherId = getUserIdFromToken(token);
116     // 校验课程/排期归属当前教师
117     if (!currentTeacherId.equals(teacherId)) {
118         throw new NoPermissionException("您无权操作其他教师的课程/排期");
119     }
120 }
121
122 /**
123  * 辅助校验: 学生操作订单时, 校验订单归属 (订单的studentId与Token中userId一致)
124  * @param token Token
125  * @param studentId 订单中的学生ID
126  */
127 public void checkStudentOwner(String token, String studentId) {
128     // 校验学生权限
129     checkStudent(token);
130     // 获取Token中的学生ID

```

```

131         String currentStudentId = getUserIdFromToken(token);
132         // 校验订单归属当前学生
133         if (!currentStudentId.equals(studentId)) {
134             throw new NoPermissionException("您无权操作他人订单");
135         }
136     }
137 }

```

1.8 启动类（完整实现，项目入口）

```

1  package com.language.reservation;
2
3  import org.mybatis.spring.annotation.MapperScan;
4  import org.springframework.boot.SpringApplication;
5  import org.springframework.boot.autoconfigure.SpringBootApplication;
6  import org.springframework.context.annotation.Bean;
7  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
8
9  /**
10   * 语言教学预约系统后端API启动类，项目唯一入口
11   * 核心关联现有文件，确保所有组件加载生效，标注如下：
12   */
13  @SpringBootApplication // 关联点1: 自动扫描当前包（com.language.reservation）及其
    子包下所有组件
14
15     // 涵盖所有controller、service、utils、exception、common
    等包下的注解类（@RestController、@Service、@Component等）
16  @MapperScan("com.language.reservation.mapper") // 关联点2: 扫描MyBatis映射接口
    (UserMapper、CourseMapper等)
17
18     // 对应1.2 application.yml中
    mybatis.type-aliases-package配置，确保mapper接口被Spring管理
19  public class LanguageReservationApplication {
20
21     public static void main(String[] args) {
22         // 启动Spring Boot项目，加载所有配置和组件（关联所有基础配置、业务模块）
23         SpringApplication.run(LanguageReservationApplication.class, args);
24     }
25
26     /**
27      * 注入BCryptPasswordEncoder对象，用于密码加密
28      * 关联点3: 对应UserService中注入的BCryptPasswordEncoder，实现密码加密逻辑（设
    计2.3 安全设计-密码加密）
29      * 若不注入，UserService中@Autowired注入会报错，无法完成用户注册、登录的密码校验
30      */
31     @Bean
32     public BCryptPasswordEncoder passwordEncoder() {

```

```
31         return new BCryptPasswordEncoder();
32     }
33 }
```

二、核心业务模块API实现

2.1 用户注册与认证模块（对应设计2.2.1 用户注册与认证模块）

2.1.1 实体类（User.java）

```
1  package com.language.reservation.entity;
2
3  import lombok.Data;
4  import javax.validation.constraints.Email;
5  import javax.validation.constraints.NotBlank;
6  import javax.validation.constraints.Pattern;
7  import javax.validation.constraints.Size;
8
9  /**
10   * 用户实体类，对应设计2.2.1 学生注册、教师注册接口的请求参数和数据库表结构
11   */
12  @Data
13  public class User {
14      private String userId; // 系统生成唯一标识（UUID），对应通用校验规则-ID类参数
15
16      // 手机号校验（对应通用校验规则-手机号）
17      @NotBlank(message = "手机号不能为空")
18      @Pattern(regexp = "^1[3456789]\\d{9}$", message = "手机号格式错误")
19      private String phone;
20
21      // 邮箱校验（对应通用校验规则-邮箱）
22      @NotBlank(message = "邮箱不能为空")
23      @Email(message = "邮箱格式错误")
24      private String email;
25
26      // 密码校验（对应通用校验规则-密码）
27      @NotBlank(message = "密码不能为空")
28      @Size(min = 8, max = 20, message = "密码长度需8-20位")
29      @Pattern(regexp = "^(?=.*[a-zA-Z])(?=.*\\d){8,20}$", message = "密码需包含字母和数字")
30      private String password;
31
32      // 角色（student/teacher/admin），对应设计2.3 安全设计-RBAC模型
33      private String role;
```

```

34
35     // 学生专属参数 (对应设计2.2.1 学生注册接口)
36     private String learnGoal;
37     private String languageLevel;    // 枚举值: 入门/进阶/中级/高级/精通
38
39     // 教师专属参数 (对应设计2.2.1 教师注册接口)
40     private String name;
41     private String qualification;    // Base64编码的资质图片
42     private String languageType;    // 枚举值: 英语/日语/韩语/法语/德语/西班牙语
43
44     // 账号状态 (active/inactive/frozen) , 对应设计2.2.1 教师注册审核、2.2.5 用户管
    理
45     private String status;
46 }

```

2.1.2 控制器 (UserController.java)

```

1  package com.language.reservation.controller;
2
3  import com.language.reservation.common.Result;
4  import com.language.reservation.entity.User;
5  import com.language.reservation.service.UserService;
6  import org.springframework.beans.factory.annotation.Autowired;
7  import org.springframework.validation.annotation.Validated;
8  import org.springframework.web.bind.annotation.PostMapping;
9  import org.springframework.web.bind.annotation.RequestBody;
10 import org.springframework.web.bind.annotation.RequestMapping;
11 import org.springframework.web.bind.annotation.RestController;
12
13 import java.util.Map;
14
15 /**
16  * 用户注册与认证控制器, 对应设计2.2.1 用户注册与认证模块所有接口
17  */
18 @RestController
19 @RequestMapping("/user")
20 @Validated
21 public class UserController {
22
23     @Autowired
24     private UserService userService;
25
26     /**
27      * 学生注册接口, 对应设计2.2.1 接口: /api/v1/user/student/register
28      */

```

```

29     @PostMapping("/student/register")
30     public Result<Map<String, String>> studentRegister(@Validated @RequestBody
User user) {
31         Map<String, String> resultMap = userService.studentRegister(user);
32         return Result.success(resultMap, "学生注册成功");
33     }
34
35     /**
36      * 教师注册接口, 对应设计2.2.1 接口: /api/v1/user/teacher/register
37      */
38     @PostMapping("/teacher/register")
39     public Result<Void> teacherRegister(@Validated @RequestBody User user) {
40         userService.teacherRegister(user);
41         return Result.success(null, "教师注册申请提交成功, 请等待管理员审核");
42     }
43
44     /**
45      * 用户登录接口, 对应设计2.2.1 接口: /api/v1/user/login
46      */
47     @PostMapping("/login")
48     public Result<Map<String, String>> login(
49         @NotBlank(message = "账号不能为空") String account,
50         @NotBlank(message = "密码不能为空") String password) {
51         Map<String, String> resultMap = userService.login(account, password);
52         return Result.success(resultMap, "登录成功");
53     }
54
55     /**
56      * 密码找回 (验证码验证) , 对应设计2.2.1 接口: /api/v1/user/password/forgot
57      */
58     @PostMapping("/password/forgot")
59     public Result<Void> forgotPassword(
60         @NotBlank(message = "账号不能为空") String account,
61         @NotBlank(message = "验证码不能为空")
62         @Pattern(regexp = "^\\d{6}$", message = "验证码格式错误") String
verifyCode) {
63         userService.verifyForgotCode(account, verifyCode);
64         return Result.success(null, "验证码验证成功, 请重置密码");
65     }
66
67     /**
68      * 密码重置, 对应设计2.2.1 接口: /api/v1/user/password/reset
69      */
70     @PostMapping("/password/reset")
71     public Result<Void> resetPassword(
72         @NotBlank(message = "账号不能为空") String account,
73         @NotBlank(message = "新密码不能为空")

```

```

74         @Pattern(regexp = "^(?=.*[a-zA-Z])(?=.*\\d){8,20}$", message =
            "新密码需包含字母和数字, 长度8-20位") String newPassword,
75         @NotBlank(message = "验证码不能为空") String verifyCode) {
76             userService.resetPassword(account, newPassword, verifyCode);
77             return Result.success(null, "密码重置成功");
78     }
79 }

```

2.1.3 服务层 (UserService.java)

```

1  package com.language.reservation.service;
2
3  import com.language.reservation.entity.User;
4  import com.language.reservation.exception.BusinessException;
5  import com.language.reservation.exception.ResourceNotFoundException;
6  import com.language.reservation.mapper.UserMapper;
7  import com.language.reservation.utils.JwtUtil;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
10 import org.springframework.stereotype.Service;
11 import org.springframework.transaction.annotation.Transactional;
12
13 import java.util.HashMap;
14 import java.util.Map;
15 import java.util.UUID;
16
17 /**
18  * 用户注册与认证服务, 对应设计2.2.1 用户注册与认证模块所有接口的业务逻辑
19  */
20 @Service
21 public class UserService {
22
23     @Autowired
24     private UserMapper userMapper;
25     @Autowired
26     private BCryptPasswordEncoder passwordEncoder;
27     @Autowired
28     private JwtUtil jwtUtil;
29
30     // 学生注册 (对应设计2.2.1 学生注册接口)
31     @Transactional
32     public Map<String, String> studentRegister(User user) {
33         // 校验手机号/邮箱是否已注册 (对应通用校验规则-手机号/邮箱唯一性)
34         if (userMapper.selectByPhone(user.getPhone()) != null) {
35             throw new BusinessException("该手机号已注册");

```

```

36         }
37         if (userMapper.selectByEmail(user.getEmail()) != null) {
38             throw new BusinessException("该邮箱已注册");
39         }
40         // 生成唯一userId (UUID) , 对应通用校验规则-ID类参数
41         user.setUserId(UUID.randomUUID().toString());
42         // 设置角色为student, 状态为active (学生注册后直接激活)
43         user.setRole("student");
44         user.setStatus("active");
45         // 密码加密 (对应设计2.3 安全设计-密码加密)
46         user.setPassword(passwordEncoder.encode(user.getPassword()));
47         // 插入数据库
48         userMapper.insert(user);
49         // 生成Token (对应设计2.3 安全设计-Token)
50         String token = jwtUtil.generateToken(user.getUserId(),
user.getRole());
51         // 组装返回数据 (对应设计2.2.1 学生注册返回数据)
52         Map<String, String> resultMap = new HashMap<>();
53         resultMap.put("userId", user.getUserId());
54         resultMap.put("token", token);
55         return resultMap;
56     }
57
58     // 教师注册 (对应设计2.2.1 教师注册接口)
59     @Transactional
60     public void teacherRegister(User user) {
61         // 校验手机号/邮箱是否已注册
62         if (userMapper.selectByPhone(user.getPhone()) != null) {
63             throw new BusinessException("该手机号已注册");
64         }
65         if (userMapper.selectByEmail(user.getEmail()) != null) {
66             throw new BusinessException("该邮箱已注册");
67         }
68         // 生成唯一userId
69         user.setUserId(UUID.randomUUID().toString());
70         // 设置角色为teacher, 状态为inactive (待管理员审核)
71         user.setRole("teacher");
72         user.setStatus("inactive");
73         // 密码加密
74         user.setPassword(passwordEncoder.encode(user.getPassword()));
75         // 插入数据库
76         userMapper.insert(user);
77     }
78
79     // 用户登录 (对应设计2.2.1 登录接口)
80     public Map<String, String> login(String account, String password) {
81         // 查找用户 (账号可为手机号/邮箱, 对应设计2.2.1 登录接口请求参数)

```



```

82         User user = userMapper.selectByPhoneOrEmail(account);
83         if (user == null) {
84             throw new ResourceNotFoundException("账号不存在");
85         }
86         // 校验密码 (对应设计2.3 安全设计-密码加密)
87         if (!passwordEncoder.matches(password, user.getPassword())) {
88             throw new BusinessException("密码错误");
89         }
90         // 校验账号状态 (对应设计2.2.1 教师注册审核、2.2.5 用户管理)
91         if ("inactive".equals(user.getStatus()) &&
            "teacher".equals(user.getRole())) {
92             throw new BusinessException("账号未审核, 请等待管理员审核");
93         }
94         if ("frozen".equals(user.getStatus())) {
95             throw new BusinessException("账号已冻结, 请联系管理员");
96         }
97         // 生成Token
98         String token = jwtUtil.generateToken(user.getUserId(),
            user.getRole());
99         // 组装返回数据 (对应设计2.2.1 登录返回数据)
100        Map<String, String> resultMap = new HashMap<>();
101        resultMap.put("userId", user.getUserId());
102        resultMap.put("role", user.getRole());
103        resultMap.put("token", token);
104        return resultMap;
105    }
106
107    // 验证码验证 (密码找回) , 对应设计2.2.1 密码找回接口
108    public void verifyForgotCode(String account, String verifyCode) {
109        // 查找用户
110        User user = userMapper.selectByPhoneOrEmail(account);
111        if (user == null) {
112            throw new ResourceNotFoundException("账号不存在");
113        }
114        // 模拟验证码校验 (实际需对接短信/邮箱接口, 有效期5分钟)
115        if (!"123456".equals(verifyCode)) { // 模拟验证码, 实际需从缓存获取
116            throw new BusinessException("验证码错误或已过期");
117        }
118    }
119
120    // 密码重置, 对应设计2.2.1 密码重置接口
121    @Transactional
122    public void resetPassword(String account, String newPassword, String
            verifyCode) {
123        // 先校验验证码
124        verifyForgotCode(account, verifyCode);
125        // 查找用户

```

```

126         User user = userMapper.selectByPhoneOrEmail(account);
127         // 校验新密码与原密码是否相同 (对应通用校验规则-密码)
128         if (passwordEncoder.matches(newPassword, user.getPassword())) {
129             throw new BusinessException("新密码不能与原密码相同");
130         }
131         // 加密新密码并更新
132         user.setPassword(passwordEncoder.encode(newPassword));
133         userMapper.updatePassword(user);
134     }
135 }

```

2.2 课程与排期管理模块（对应设计2.2.2 课程与排期管理模块）

2.2.1 核心实体类（CourseTemplate.java、Course.java、Schedule.java）

```

1  package com.language.reservation.entity;
2
3  import lombok.Data;
4  import javax.validation.constraints.NotBlank;
5  import javax.validation.constraints.NotNull;
6  import javax.validation.constraints.Size;
7  import java.math.BigDecimal;
8
9  // 课程模板实体（对应设计2.2.2 课程模板相关接口）
10 @Data
11 public class CourseTemplate {
12     private String templateId; // 唯一标识 (UUID)
13     @NotBlank(message = "语言类型不能为空")
14     private String languageType; // 枚举值：英语/日语/韩语/法语/德语/西班牙语
15     @NotBlank(message = "难度等级不能为空")
16     private String difficultyLevel; // 枚举值：入门/进阶/中级/高级
17     @NotNull(message = "课时费不能为空")
18     private BigDecimal classFee; // ≥0, 保留2位小数 (对应通用校验规则)
19     @NotNull(message = "课程时长不能为空")
20     private Integer classDuration; // ≥15, 15的倍数 (对应通用校验规则)
21     @NotBlank(message = "课程形式不能为空")
22     private String classForm; // 枚举值：一对一/小班课/大班课
23     @NotBlank(message = "课程描述不能为空")
24     @Size(min = 10, max = 500, message = "课程描述长度需10-500字")
25     private String description;
26 }
27
28 // 教师课程实体（对应设计2.2.2 教师课程相关接口）
29 @Data
30 public class Course {

```

```

31     private String courseId; // 唯一标识 (UUID)
32     @NotBlank(message = "模板ID不能为空")
33     private String templateId; // 关联课程模板
34     @NotBlank(message = "课程名称不能为空")
35     @Size(min = 2, max = 50, message = "课程名称长度需2-50字")
36     private String courseName;
37     @NotBlank(message = "教学内容不能为空")
38     @Size(min = 10, max = 1000, message = "教学内容长度需10-1000字")
39     private String content;
40     @NotBlank(message = "课程特色不能为空")
41     @Size(min = 10, max = 1000, message = "课程特色长度需10-1000字")
42     private String feature;
43     @NotBlank(message = "教师ID不能为空")
44     private String teacherId; // 关联教师
45 }
46
47 // 课程排期实体 (对应设计2.2.2 排期相关接口)
48 @Data
49 public class Schedule {
50     private String scheduleId; // 唯一标识 (UUID)
51     @NotBlank(message = "课程ID不能为空")
52     private String courseId; // 关联教师课程
53     @NotBlank(message = "开始时间不能为空")
54     private String startTime; // 格式YYYY-MM-DD HH:mm:ss (对应通用校验规则-时间)
55     @NotBlank(message = "结束时间不能为空")
56     private String endTime;
57     @NotNull(message = "重复标识不能为空")
58     private Boolean isRepeat; // 是否重复 (每周)
59     private Integer repeatWeek; // 1-7, isRepeat=true时必填 (对应通用校验规则)
60     private String status; // 枚举值: available/unavailable (可预约/不可预约)
61 }

```

2.2.2 控制器 (CourseController.java)

```

1  package com.language.reservation.controller;
2
3  import com.language.reservation.common.Result;
4  import com.language.reservation.entity.Course;
5  import com.language.reservation.entity.CourseTemplate;
6  import com.language.reservation.entity.Schedule;
7  import com.language.reservation.service.CourseService;
8  import com.language.reservation.utils.PermissionCheck;
9  import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.validation.annotation.Validated;

```

```
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14 import java.util.Map;
15
16 /**
17  * 课程与排期管理控制器，对应设计2.2.2 课程与排期管理模块所有接口
18  */
19 @RestController
20 @RequestMapping("/course")
21 @Validated
22 public class CourseController {
23
24     @Autowired
25     private CourseService courseService;
26     @Autowired
27     private PermissionCheck permissionCheck;
28
29     /**
30      * 创建课程模板，对应设计2.2.2 接口：/api/v1/course/template/add（管理员权限）
31      */
32     @PostMapping("/template/add")
33     public Result<Map<String, String>> addTemplate(
34         @Validated @RequestBody CourseTemplate template,
35         @RequestHeader("Authorization") String token) {
36         // 权限校验：仅管理员可操作
37         permissionCheck.checkAdmin(token);
38         Map<String, String> resultMap = courseService.addTemplate(template);
39         return Result.success(resultMap, "课程模板创建成功");
40     }
41
42     /**
43      * 教师创建课程，对应设计2.2.2 接口：/api/v1/course/teacher/add（教师权限）
44      */
45     @PostMapping("/teacher/add")
46     public Result<Map<String, String>> addCourse(
47         @Validated @RequestBody Course course,
48         @RequestHeader("Authorization") String token) {
49         // 权限校验：仅教师可操作
50         permissionCheck.checkTeacher(token);
51         // 校验课程归属当前教师
52         String teacherId = permissionCheck.getUserIdFromToken(token);
53         course.setTeacherId(teacherId);
54         Map<String, String> resultMap = courseService.addCourse(course);
55         return Result.success(resultMap, "课程创建成功");
56     }
57 }
```

```

58     /**
59      * 设置课程排期, 对应设计2.2.2 接口: /api/v1/course/schedule/set (教师权限)
60      */
61     @PostMapping("/schedule/set")
62     public Result<Void> setSchedule(
63         @Validated @RequestBody Schedule schedule,
64         @RequestHeader("Authorization") String token) {
65         // 权限校验: 仅教师可操作
66         permissionCheck.checkTeacher(token);
67         // 校验排期归属当前教师
68         String teacherId = permissionCheck.getUserIdFromToken(token);
69         courseService.checkScheduleOwner(schedule.getCourseId(), teacherId);
70         courseService.setSchedule(schedule);
71         return Result.success(null, "排期设置成功");
72     }
73
74     /**
75      * 更新课程排期, 对应设计2.2.2 接口: /api/v1/course/schedule/update (教师权限)
76      */
77     @PutMapping("/schedule/update")
78     public Result<Void> updateSchedule(
79         @Validated @RequestBody Schedule schedule,
80         @RequestHeader("Authorization") String token) {
81         // 权限校验: 仅教师可操作
82         permissionCheck.checkTeacher(token);
83         // 校验排期归属当前教师
84         String teacherId = permissionCheck.getUserIdFromToken(token);
85         courseService.checkScheduleOwner(schedule.getScheduleId(), teacherId);
86         courseService.updateSchedule(schedule);
87         return Result.success(null, "排期更新成功");
88     }
89
90     /**
91      * 查询课程模板列表, 对应设计2.2.2 接口: /api/v1/course/template/list (教师、管
92      理员权限)
93      */
94     @GetMapping("/template/list")
95     public Result<Map<String, List<CourseTemplate>>> getTemplateList(
96         String languageType,
97         @RequestHeader("Authorization") String token) {
98         // 权限校验: 教师或管理员可操作
99         permissionCheck.checkTeacherOrAdmin(token);
100         List<CourseTemplate> templates =
101             courseService.getTemplateList(languageType);
102         return Result.success(Map.of("templates", templates), "查询成功");
103     }
104 }

```

2.2.3 服务层 (CourseService.java)

```
1  package com.language.reservation.service;
2
3  import com.language.reservation.entity.Course;
4  import com.language.reservation.entity.CourseTemplate;
5  import com.language.reservation.entity.Schedule;
6  import com.language.reservation.exception.BusinessException;
7  import com.language.reservation.exception.ResourceNotFoundException;
8  import com.language.reservation.mapper.CourseMapper;
9  import com.language.reservation.mapper.ScheduleMapper;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.text.ParseException;
15 import java.text.SimpleDateFormat;
16 import java.util.Date;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.UUID;
20
21 /**
22  * 课程与排期管理服务，对应设计2.2.2 课程与排期管理模块所有接口的业务逻辑
23  * 涵盖课程模板、教师课程、课程排期的增删改查，严格遵循权限校验和数据校验规则
24  */
25 @Service
26 public class CourseService {
27
28     @Autowired
29     private CourseMapper courseMapper;
30     @Autowired
31     private ScheduleMapper scheduleMapper;
32
33     // 日期格式化工具，对应通用校验规则-时间格式 (YYYY-MM-DD HH:mm:ss)
34     private static final SimpleDateFormat DATE_FORMAT = new
SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
35
36     /**
37      * 创建课程模板，对应设计2.2.2 课程模板创建接口，仅管理员可操作
38      */
39     @Transactional
40     public Map<String, String> addTemplate(CourseTemplate template) {
41         // 校验课程模板唯一性 (语言类型+难度等级唯一，对应通用校验规则-唯一性)
```

```

42         if
(courseMapper.selectTemplateByLangAndLevel(template.getLanguageType(),
template.getDifficultyLevel()) != null) {
43             throw new BusinessException("该语言类型+难度等级的课程模板已存在");
44         }
45         // 生成唯一模板ID (UUID) , 对应通用校验规则-ID类参数
46         String templateId = UUID.randomUUID().toString();
47         template.setTemplateId(templateId);
48         // 插入课程模板到数据库
49         courseMapper.insertTemplate(template);
50         // 组装返回数据 (对应设计2.2.2 模板创建返回数据)
51         Map<String, String> resultMap = Map.of("templateId", templateId);
52         return resultMap;
53     }
54
55     /**
56      * 教师创建课程, 对应设计2.2.2 教师课程创建接口, 仅教师可操作
57      */
58     @Transactional
59     public Map<String, String> addCourse(Course course) {
60         // 校验课程模板是否存在 (对应设计2.2.2 课程关联模板规则)
61         CourseTemplate template =
courseMapper.selectTemplateById(course.getTemplateId());
62         if (template == null) {
63             throw new ResourceNotFoundException("课程模板不存在, 请先选择正确的模
板");
64         }
65         // 生成唯一课程ID (UUID)
66         String courseId = UUID.randomUUID().toString();
67         course.setCourseId(courseId);
68         // 插入教师课程到数据库
69         courseMapper.insertCourse(course);
70         // 组装返回数据 (对应设计2.2.2 课程创建返回数据)
71         Map<String, String> resultMap = Map.of("courseId", courseId);
72         return resultMap;
73     }
74
75     /**
76      * 设置课程排期, 对应设计2.2.2 排期设置接口, 仅教师可操作
77      */
78     @Transactional
79     public void setSchedule(Schedule schedule) {
80         // 1. 校验课程是否存在
81         Course course = courseMapper.selectCourseById(schedule.getCourseId());
82         if (course == null) {
83             throw new ResourceNotFoundException("课程不存在, 无法设置排期");
84         }

```

```

85         // 2. 校验时间格式和逻辑 (对应通用校验规则-时间)
86         validateScheduleTime(schedule.getStartTime(), schedule.getEndTime());
87         // 3. 校验重复排期规则 (isRepeat=true时, repeatWeek必填且为1-7)
88         if (schedule.getIsRepeat()) {
89             if (schedule.getRepeatWeek() == null || schedule.getRepeatWeek()
90 < 1 || schedule.getRepeatWeek() > 7) {
91                 throw new BusinessException("重复排期需指定每周重复日期 (1-7, 对应
92 周一至周日)");
93             }
94         } else {
95             // 非重复排期, 校验该时间段是否已存在排期 (避免冲突)
96             if (scheduleMapper.selectScheduleByTime(schedule.getCourseId(),
97 schedule.getStartTime(), schedule.getEndTime()) != null) {
98                 throw new BusinessException("该时间段已存在排期, 请勿重复设置");
99             }
100         }
101         // 4. 生成唯一排期ID, 设置默认状态为available (可预约)
102         String scheduleId = UUID.randomUUID().toString();
103         schedule.setScheduleId(scheduleId);
104         schedule.setStatus("available");
105         // 5. 插入排期到数据库
106         scheduleMapper.insertSchedule(schedule);
107     }
108
109     /**
110      * 更新课程排期, 对应设计2.2.2 排期更新接口, 仅教师可操作
111      */
112     @Transactional
113     public void updateSchedule(Schedule schedule) {
114         // 1. 校验排期是否存在
115         Schedule existingSchedule =
116 scheduleMapper.selectScheduleById(schedule.getScheduleId());
117         if (existingSchedule == null) {
118             throw new ResourceNotFoundException("排期不存在, 无法更新");
119         }
120         // 2. 校验时间格式和逻辑 (若更新时间)
121         if (schedule.getStartTime() != null && schedule.getEndTime() != null)
122     {
123         validateScheduleTime(schedule

```

(注: 文档部分内容可能由 AI 生成)